



C2

Build-vs-buy — and the DBS case

12 min capsule + live case

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Decide, layer by layer, what an AI product should own versus rent — and read those decisions in a real bank.

Build-vs-buy: five factors

One decision, asked at every layer of the stack.

Cost — build cheaper at scale? rent cheaper to start?

Latency — can a vendor hit your response-time budget?

Control — how much must you tune, inspect, guarantee?

Data gravity — can the data even leave your estate?

Regulation — what must be auditable / on-shore?

Data gravity, for the builders: *moving TB–PB of data costs egress \$ + latency, so you bring compute to the data; regulated / residency-bound data legally can't move at all → on-prem or in-region.*

Takeaway Own your moat and your risk; rent the commodity.

The stack, tagged build-vs-buy

A typical enterprise answer — yours will differ by context.

Layer	Usually...	Why
Compute	Rent	clouds do it cheaper than you
Data	Build/own	it's your moat; often can't move it
Vector store	Buy	mature, commodity
Model	Rent	frontier models cost billions to train
Orchestration	Build	this is your product logic
Serving	Mix	buy infra, tune for your latency
Evals / governance	Build/own	your risk, your accountability

Takeaway Own data, orchestration, governance; rent compute and the model.

POLL · WOOCCLAP

For a bank, which factor should win most often?

Cost

Latency

Control

Data gravity

Regulation

'Data gravity' = data too heavy or regulated to leave your estate. Keep your pick — compare it to DBS.

Build-vs-buy differs by who you are

Same five factors — plus the big axis: open-weight (self-host) vs closed / API models.

Who	Build-vs-buy stance	Models & hosting
Regulated bank (DBS)	Own data + governance; rent the model	Closed API; open-weight for sensitive flows
Startup · non-regulated	Buy everything (hosted API); ship fast	Closed API now; cost-tune later
Startup · regulated / mission-critical	Own data + controls even while tiny	Open-weight self-host or in-region API
Hospital · defence · utilities	On-prem; data cannot leave	Open-weight only; air-gapped if needed
Sovereign / national AI	Own the stack in-country	Open-weight, trained / hosted domestically
Telecom (Jio / Singtel)	Fine-tune for cost at scale	Open-weight fine-tuned on own logs

Takeaway The more regulated or mission-critical, the more you lean open-weight + self-hosted (own the data); everyone else rents closed APIs for speed.

The DBS case — how we'll run it

You have the pre-read. We brief live. The full case is released after class.

Pre-read (done) → Live brief (now, ~15 min) → Team discussion + submit (~15 min) → Share (~10 min) → Full case released post-class

Case: DBS' AI Journey — Harvard Business School 625-053 (Zhu, Zhu & Wong).

Anchor question for today: which layers did a regulated bank build, and which did it rent — and why?

DBS — the situation

May 2024, a record quarter — and a two-part decision: how far to trust GenAI, and which layers of the stack to own vs rent.

- Singapore's largest bank; once mocked 'Damn Bloody Slow' — now a digital leader.
- AI since 2014: a failed IBM Watson bet, then a decade of disciplined build.
- GenAI is live internally and promising — but nascent and error-prone.
- Piyush's call: which use cases to scale and how to keep them safe — AND which layers of the AI stack to build vs buy.

Why this is our case: it exercises Class 2 directly — read the modern AI stack (ILO 1) and defend build-vs-buy per layer (ILO 3). Watch which layers DBS owns vs rents.

Takeaway An incumbent that industrialised AI — now choosing how far to trust GenAI AND which layers of the stack to own.

The stack DBS built

Own the data and the governance; rent the models.

ADA — the data layer

Advancing DBS with AI.

- 12 warehouses → one 'source of truth'
- Tiered data; role-based instant access
- What took months now takes seconds

P-U-R-E — the governance layer

Purposeful · Unsurprising · Respectful · Explainable.

- Every use case must clear it
- RDU Committee 'white-lists' approvals
- Move fast — within guardrails

Takeaway DBS built the two layers a bank can't outsource: data and governance.

The GenAI bet — one horizontal, three verticals

The horizontal IS the Class-2 stack (grounded in DBS); the verticals build on it.

Call-centre co-pilot

- Summarise calls; draft replies
- Pulls product/policy from EKB
- Projected +20% productivity

RM co-pilot

- Client research + portfolio prep
- AML / KYC screening

Software development

- Code generation
- Testing / QA assist

▲ *the three verticals run on the one platform below* ▲

Horizontal — DBS-GPT platform *the Class-2 stack — a DEEP platform beneath a mile-wide, inch-deep assistant*

Model: rented GPT-4 / Claude **Data:** ADA (backbone · sandbox · factory) **Retrieval:** EKB = RAG **Orchestration:** DBS-GPT (via GSTAR) **Governance:** P-U-R-E + RDU **Serving:** internal-only, human-in-the-loop

Contrast: *how did Moderna (Class 1) govern GenAI differently? → next slide.*

DBS vs Moderna — two ways to build & govern GenAI

Callback to Class 1: two orgs, opposite choices across the stack and its governance.

Layer / decision	Moderna (Class 1)	DBS
Models	Rent closed — OpenAI (ChatGPT Enterprise)	Rent closed — GPT-4 / Claude, wrapped in RAG
Data	Employee GPTs over varied data; broad access	One 'source of truth' (ADA); tiered, role-based
Retrieval	Files uploaded per custom GPT	Enterprise Knowledge Base (EKB) = RAG
Orchestration / build	Buy the platform; staff assemble GPTs	Build DBS-GPT + controls in-house
Governance	Bottom-up; central control catching up (post-leak)	Top-down: RDU white-list + P-U-R-E gate
Regulation / depth	Pharma; mile-wide, inch-deep; lower stakes	Finance (MAS); internal-only, human-in-loop

Takeaway Same rented models — opposite stack + governance choices; how much you own and centralise scales with your regulatory stakes.

The decision on the table

Two decisions, not one: how far to trust GenAI, and which layers of the stack to own vs rent.

Trust / scope — can P-U-R-E + human-in-the-loop scale GenAI with minimal oversight — internal-only, or customer-facing accepting rare errors?

Stack / build-vs-buy — which layers to own (data, governance) vs rent (the model) — and where does the moat sit?

Workforce — how does the GenAI push reshape roles?

“When the music is on, you have to get on the dance floor.”

→ **Teams interrogate both axes next** — *three questions, pick one.*

Discuss in teams — then share

~15 min in teams, submit, then 4–5 teams share (~10 min). No wrong answers — reason about the stack.

- 1 DBS built its own data platform (ADA) and governance (P-U-R-E) but rents its foundation models. Go layer by layer — what did DBS own versus rent, and why does that split make sense for a bank?
- 2 Every DBS model must pass P-U-R-E — Purposeful, Unsurprising, Respectful, Explainable — before deployment. Which of the four is hardest for a generative-AI system to satisfy, and why?
- 3 DBS-GPT plugs the same models anyone can rent into the bank's own knowledge base. What could a rival not copy just by calling the same API — and what does that tell you about where value sits?

Bring it home: spot the same pattern — own data + governance, rent the model, a data moat — in a bank or institution from your own country. China: ICBC · CCB · WeBank · Ant · Ping An · India: HDFC · SBI · ICICI · Jio Financial

C2 · TAKEAWAY

Build-vs-buy is a per-layer decision — DBS owned data + governance, rented the models.

After the break: how retrieval actually works — then you build it.